

Ejercicios evaluables UA3

Esta actividad aborda los criterios de evaluación del RA.

Instrucciones

● Todas las actividades que se realicen deben entregarse mediante un enlace al repositorio de Github y un documento (si se pide alguna evidencia o se plantean preguntas):

DAW_ OPT_UA3_ApellidosNombre.pdf

- Las entregas deberán realizarse en la tarea correspondiente en el formato especificado antes de la hora de finalización. Cualquier tarea que no cumpla estos requisitos tendrá una calificación de 0.
- Se pedirá que defiendas tu práctica. Si no puedes justificar tu trabajo o responder las preguntas que se te plantean, la calificación de la tarea será de 0.

Sobre el uso de IA y fuentes externas

Puedes utilizar herramientas como buscadores o inteligencia artificial como apoyo, pero no como sustituto de tu trabajo. Las respuestas deben estar redactadas por ti, con tus propias ideas y razonamientos.

Se valorará especialmente el análisis personal y la justificación razonada.

Para Python/Django

Ejercicio 1

El backend incluye una vista accesible mediante la ruta `/api/health/`, cuya función es indicar si el sistema está operativo. Primero es necesario entender con claridad cuál es su comportamiento actual.

- ☐ Arranca el servidor.
- ☐ Realiza una petición manual a la ruta `/api/health/` con Bruno.
- ☐ Comprueba el código de estado HTTP devuelto.
- ☐ Analiza la estructura del JSON de respuesta.
- ☐ Anota cuál es el comportamiento exacto que consideras correcto para esta vista.

Ejercicio 2

Una comprobación completa no se limita únicamente al caso en el que todo funciona correctamente. También es importante observar cómo se comporta el sistema ante usos incorrectos o cambios inesperados.

- ☐ Realiza una petición a `/api/health/` utilizando un método HTTP distinto de GET.
- ☐ Observa el código de estado que devuelve el servidor.
- ☐ Modifica temporalmente el código de la vista para alterar su respuesta (por ejemplo, cambiando el contenido del JSON).
- ☐ Repite la petición y observa el nuevo resultado.
- ☐ Restaura el código original una vez finalizadas las comprobaciones.

Ejercicio 3

Además de las vistas, el backend incluye lógica interna en los modelos que también debe comportarse de forma predecible. Antes de automatizar estas comprobaciones, es importante observar su comportamiento manualmente.

- ☐ Accede a la consola de Django (*`python manage.py shell`*).
- ☐ Crea instancias del modelo `LibraryEntry` con distintos valores.
- ☐ Ejecuta manualmente los métodos:
 - ☐ `external_id_length()`
 - ☐ `external_id_upper()`
 - ☐ `hours_played_label()`
 - ☐ `status_value()`
- ☐ Anota qué devuelve cada método en distintos casos y qué resultados consideras correctos.

Ejercicio 4

Se te proporciona la estructura de tests del proyecto ya creada, junto con un test inicial que siempre pasa y varios tests ya implementados sobre el método `external_id_length()`. En este ejercicio se observa cómo una comprobación manual pasa a estar protegida por un test automático.

- ☐ Ejecuta la batería completa de tests del proyecto.
- ☐ Comprueba que todos los tests pasan correctamente.
- ☐ Modifica temporalmente el código del método `external_id_length()` para que devuelva un valor incorrecto.

- ☐ Vuelve a ejecutar los tests y observa qué ocurre.
- ☐ Restaura el código original del método.

Ejercicio 5

El modelo `LibraryEntry` contiene otros métodos cuyo comportamiento también es importante proteger frente a cambios futuros. En este ejercicio se empieza a ampliar la batería de tests unitarios del proyecto.

- ☐ Escribe tests unitarios para los métodos:
 - ☐ `external_id_upper()`
 - ☐ `hours_played_label()`
 - ☐ `status_value()`
- ☐ Para cada método, comprueba al menos un caso correcto.
- ☐ Incluye casos límite o poco habituales cuando tengan sentido.
- ☐ Asegúrate de que los tests expresan claramente qué comportamiento se considera válido.

Ejercicio 6

Las comprobaciones manuales realizadas al inicio de la sesión sobre la ruta `/api/health/` deben convertirse ahora en una comprobación automática que se ejecute siempre de la misma forma.

- ☐ Crea un test que realice una petición GET a `/api/health/`.
- ☐ Comprueba que el código de estado HTTP es el esperado.
- ☐ Verifica que la respuesta contiene un JSON con la estructura correcta.
- ☐ Comprueba que el test falla si se altera el comportamiento de la vista.

Ejercicio 7

Además del caso correcto, una vista también debe responder adecuadamente cuando se utiliza de forma incorrecta. Este comportamiento también debe quedar reflejado en los tests.

- ☐ Crea un test automático que realice una petición a `/api/health/` utilizando un método HTTP distinto de GET.
- ☐ Comprueba que el servidor devuelve el código de error correspondiente.
- ☐ Asegúrate de que el test deja claro que ese uso de la ruta no es válido.

Para Java

Ejercicio 1

En este primer ejercicio quiero que ejecutes el proyecto y pruebes todas las rutas que se detallan en el README.md.

Adjunta capturas que demuestren que has consultado, creado y actualizado los datos.

Se deben ejecutar los tests existentes siguiendo las instrucciones que hay en el README.md (deben existir 4 tests que deben pasar).

Ejercicio 2

Test 1: TitleRulesTest

Crear archivo: src/test/java/TitleRulesTest.java

Debe tener exactamente 4 tests, con estos nombres (obligatorio):

1. normalizeTitle_recortaYReduceEspacios
 - Entrada con espacios múltiples y extremos.
 - Esperado: un solo espacio entre palabras y sin espacios al principio/fin.
2. validateTitle_fallaSiVacio
 - Entrada: " "
 - Esperado: lanza IllegalArgumentException
3. validateTitle_fallaSiDemasiadoLargo
 - Entrada: string de 61 caracteres
 - Esperado: lanza excepción.
4. validateTitle_okSiCorrecto
 - Entrada válida
 - Esperado: no lanza excepción.

Rúbrica de evaluación de la tarea

Criterio de evaluación	Nivel alto (9-10)	Nivel medio (5-9)	Nivel Bajo (1-5)	No logrado (0)